

Analyse des performances du parallélisme

Rendu Devoir Vélo –2025 – Systèmes et Logiciels de Base –Mamadou G Diallo

1^{er} janvier 2026

1 Documenter les performances et les limites du parallélisme

Les résultats expérimentaux montrent que l'exécution parallèle par `multiprocessing` n'apporte pas de gain de performance dans ce cas précis. Le temps d'exécution séquentiel mesuré est de **0.054 s**, tandis que le temps d'exécution parallèle atteint **0.063 s**, ce qui correspond à un speedup de **0.86×**.

Cette absence d'accélération s'explique par le fait que la charge de calcul associée à chaque simulation est trop faible pour compenser les coûts induits par le parallélisme. Bien que le `multiprocessing` permette d'éviter les limitations du GIL de Python, son efficacité dépend fortement de la granularité des tâches. Dans ce contexte, le parallélisme devient contre-productif.

2 Analyser les goulets d'étranglement

Le principal goulet d'étranglement identifié est le surcoût lié à l'infrastructure du parallélisme. Celui-ci inclut la création et la gestion des processus, la sérialisation des données échangées ainsi que la communication inter-processus via les files (`Queue`).

La granularité trop fine des tâches accentue ce phénomène : le temps nécessaire à la distribution et à la collecte des résultats dépasse le temps de calcul effectif des simulations. Le calcul en lui-même n'est donc pas limitant ; ce sont les mécanismes de parallélisation qui constituent le facteur bloquant dans cette configuration.

3 Documenter les performances distribuées (MPI)

L'exécution distribuée à l'aide de MPI montre une amélioration significative des performances par rapport à l'exécution séquentielle. Le temps d'exécution séquentiel est de **0.077 s**, tandis que le temps d'exécution MPI est réduit à **0.038 s**, ce qui correspond à un speedup de **2.06×**.

Ce gain confirme que MPI est bien adapté à la distribution de tâches indépendantes sur plusieurs processus, et potentiellement sur plusieurs nœuds. Contrairement au `multiprocessing` local, MPI permet une meilleure exploitation du parallélisme lorsque la charge de travail est correctement répartie.

Néanmoins, le speedup reste inférieur au parallélisme idéal. Les principales limitations proviennent de la latence des communications, de la sérialisation des données échangées entre processus et de l'architecture maître-travailleurs, dans laquelle le processus maître peut devenir un point de congestion. Ces facteurs limitent la scalabilité lorsque le nombre de processus augmente.